



Universidad
Zaragoza

Sistemas Inteligentes 1

Resolución Problemas

Búsquedas Informadas

Índice

- Informada= Utiliza conocimiento problema
- Estrategias
 - Búsqueda primero el mejor y sus variantes (A^*)
- Función heurística

Descripción informal de la búsqueda en grafo

function BÚSQUEDA-GRAFO(*problema*, *estrategia*) **returns** solución o fallo

Inicializa la *frontera* utilizando el estado inicial del problema

Inicializa el conjunto de nodos **explorados** a vacío

loop do

if la *frontera* está vacía **then return** fallo

elige un *nodo* hoja de la frontera de acuerdo a una *estrategia*

if el *nodo* contiene un nodo objetivo **then return** solución

añade el *nodo* al conjunto de nodos **explorados**

expande el nodo elegido, añadiendo los nodos resultantes a la

frontera

sólo si no está en la *frontera* o en el conjunto **explorados**

Una *estrategia* se define por el *orden de expansión* de nodos

Búsqueda primero el mejor

Idea: Utilizar una función de evaluación de **cada nodo** $f(n)$ que estima como de “prometedor” es el nodo.

Aproximación general de la búsqueda informada

- **$f(n)$** es una estimación del coste. Elegiremos el nodo con menor coste estimado basado en la función **$f(n)$**
- La elección de **$f(n)$** determina la **estrategia**
- La mayoría de los algoritmos primero el mejor incluyen una función heurística, que denominaremos **$h(n)$** :

$h(n)$ = *estimación del coste del camino menos costoso desde el estado en el nodo n al objetivo. Depende sólo del **estado** en n .*

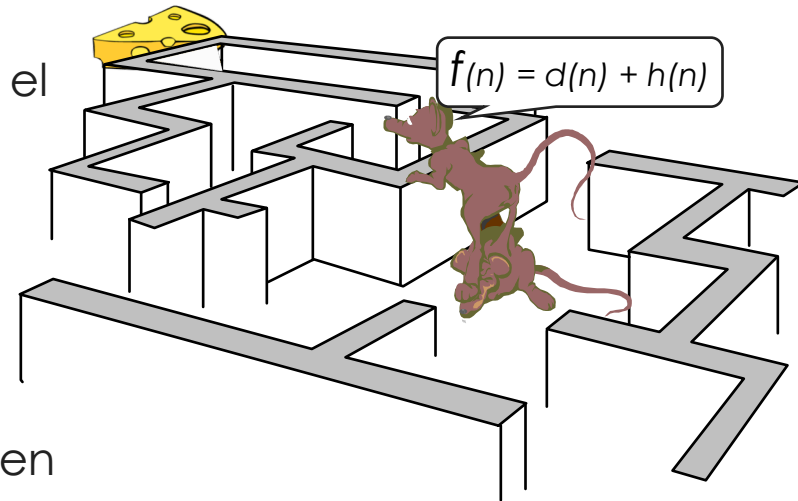
Búsqueda primero el mejor

La carga de **conocimiento** del problema viene dada por **$h(n)$**

Si no utilizamos $h(n)$ la búsqueda primero el mejor incluye a la BFS, DFS y coste uniforme.

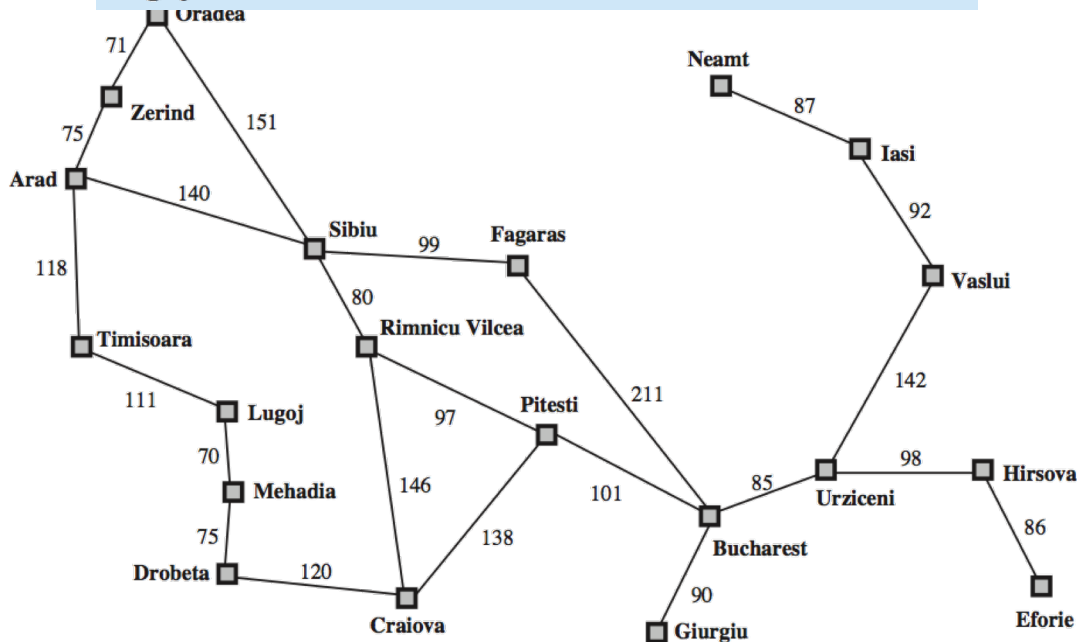
$h(n)$ es función del estado.

- **$f(n)$** es una función del nodo, recoge información del camino recorrido (nodo)
- **$h(n)$** sólo estima lo prometedor del nodo en función del estado.
- **$h(n)$** es una función no negativa, específica del problema, y con **$h(n)=0$** si el estado del nodo es un objetivo.



Ejemplo heurística: Distancia en línea recta al objetivo

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



- h_{SLD} = heurística de la línea recta al objetivo / straight-line distance heuristic.
- h_{SLD} no puede ser calculada a partir de la formulación del problema.
- No hemos introducido las coordenadas en la formulación del problema

Algoritmo voraz /Greedy algorithm

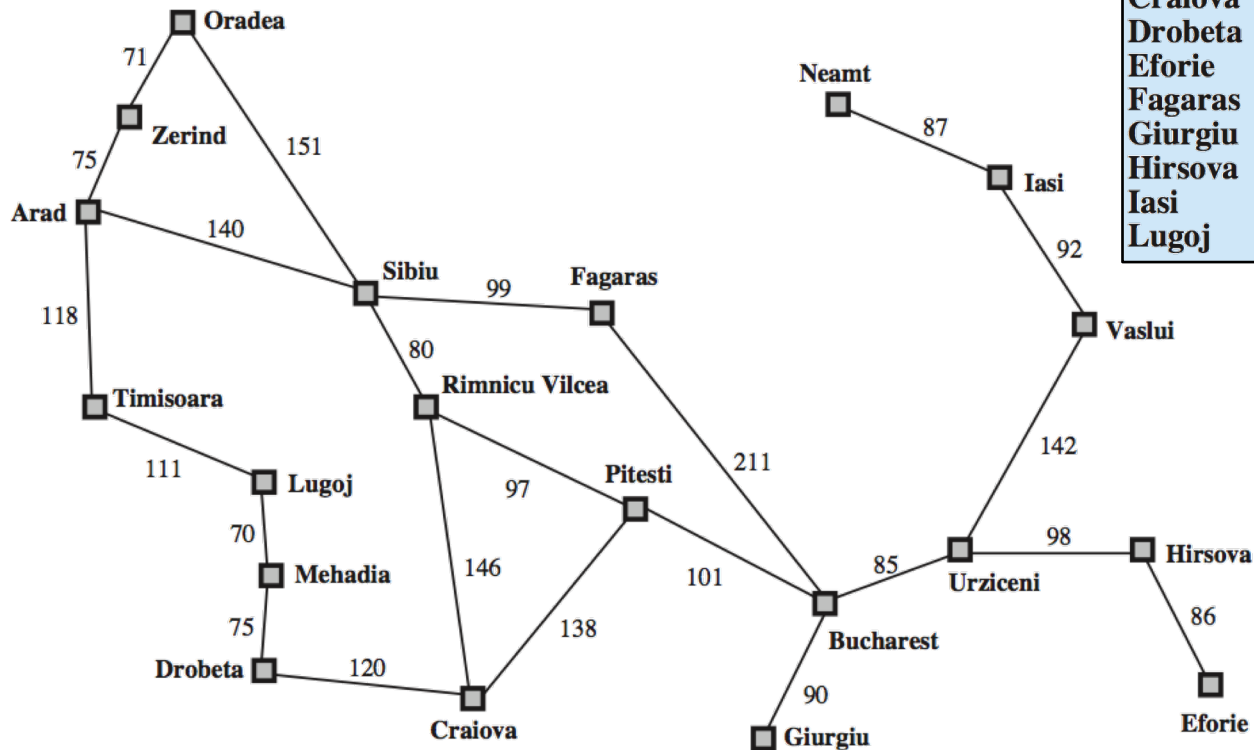
- Evaluación de función $f(n) = h(n)$ (heurística)
= estimación del coste de n al objetivo más próximo
- El algoritmo voraz expande el nodo que parece estar más próximo al objetivo.

P. e. $h_{SLD}(n)$ = distancia en línea recta de n a Bucharest

Ejemplo Algoritmo voraz

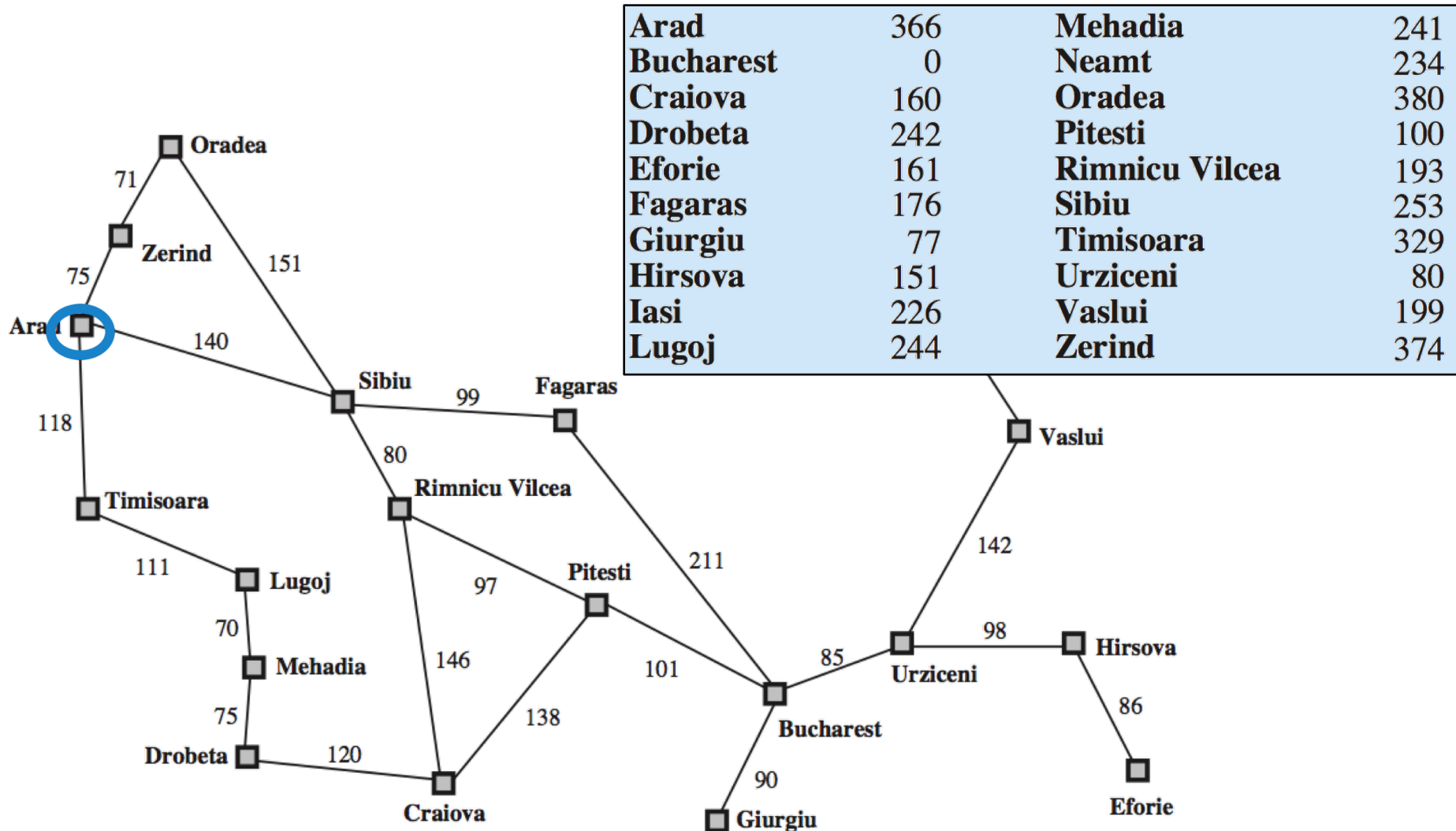
Queremos viajar de Arad a Bucharest.

- Estado Inicial = Arad

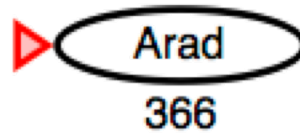


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

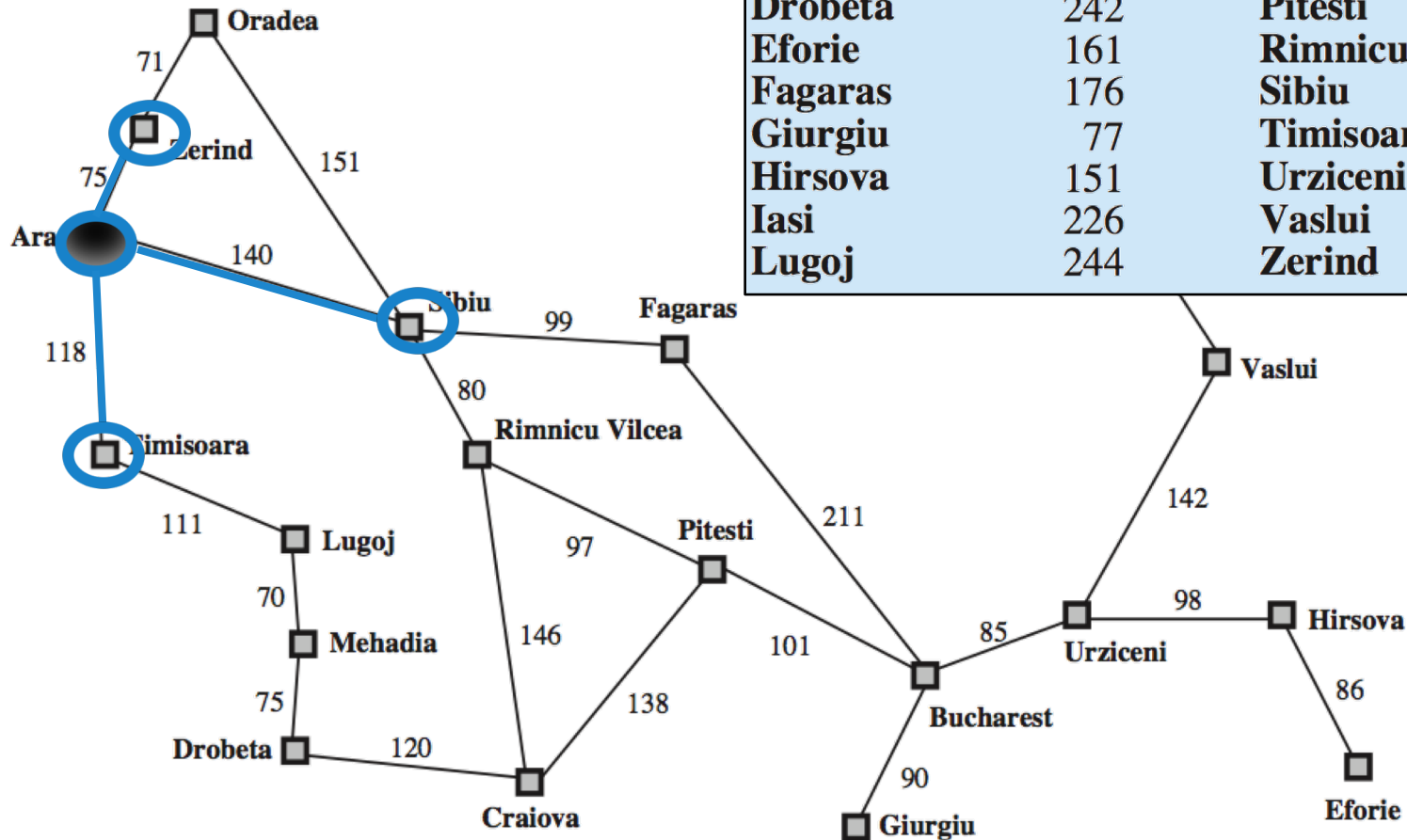
Ejemplo Algoritmo voraz (en árbol)



Ejemplo Algoritmo voraz

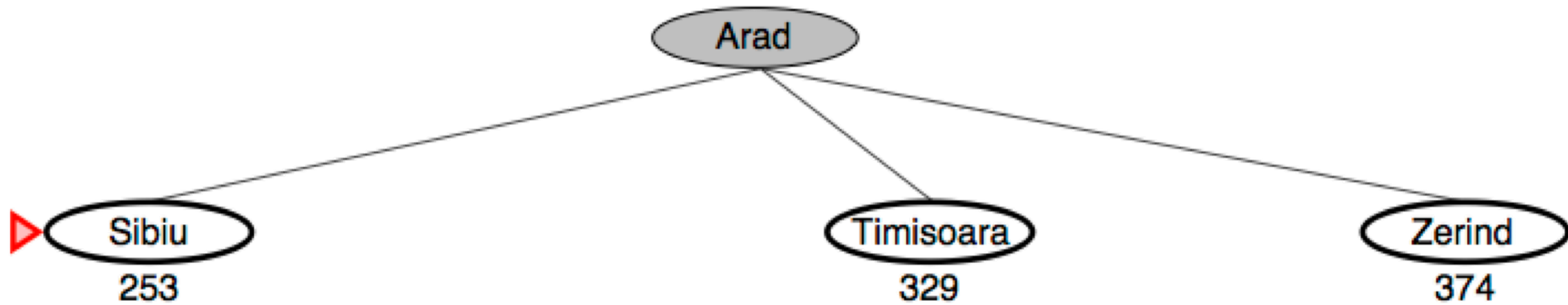


Ejemplo Algoritmo voraz

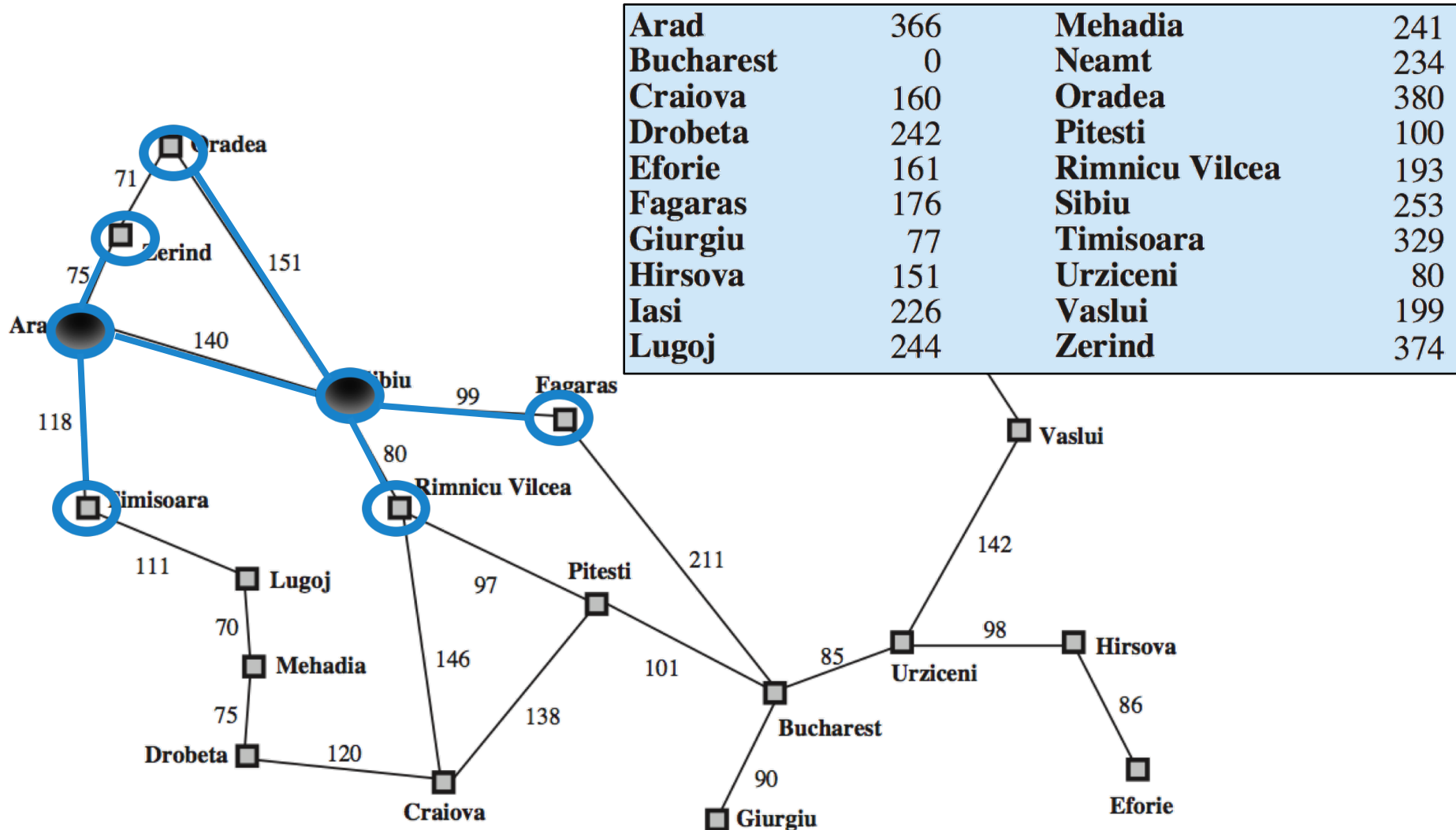


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

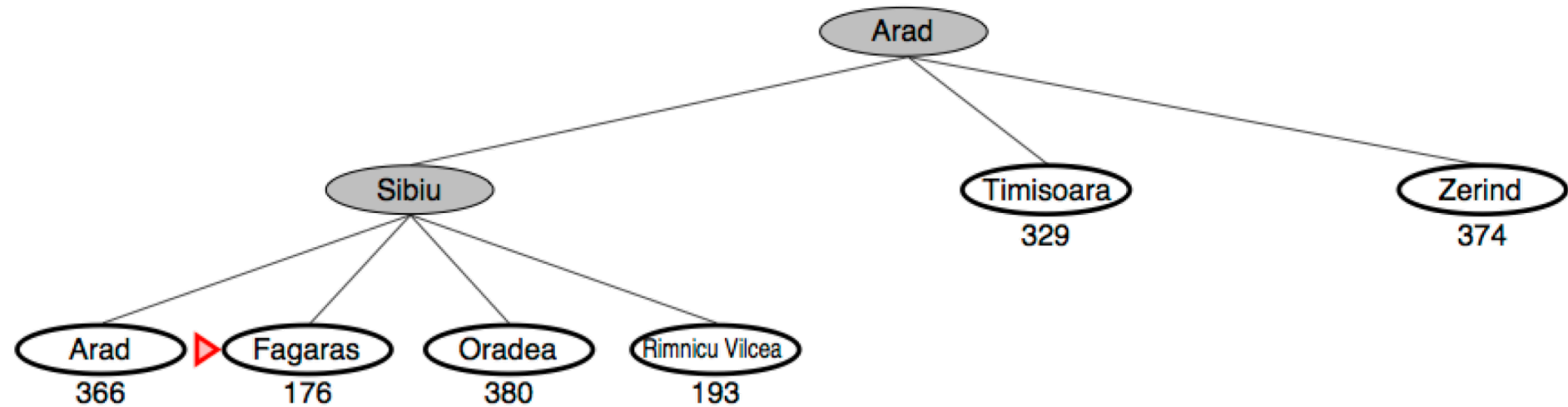
Ejemplo Algoritmo voraz



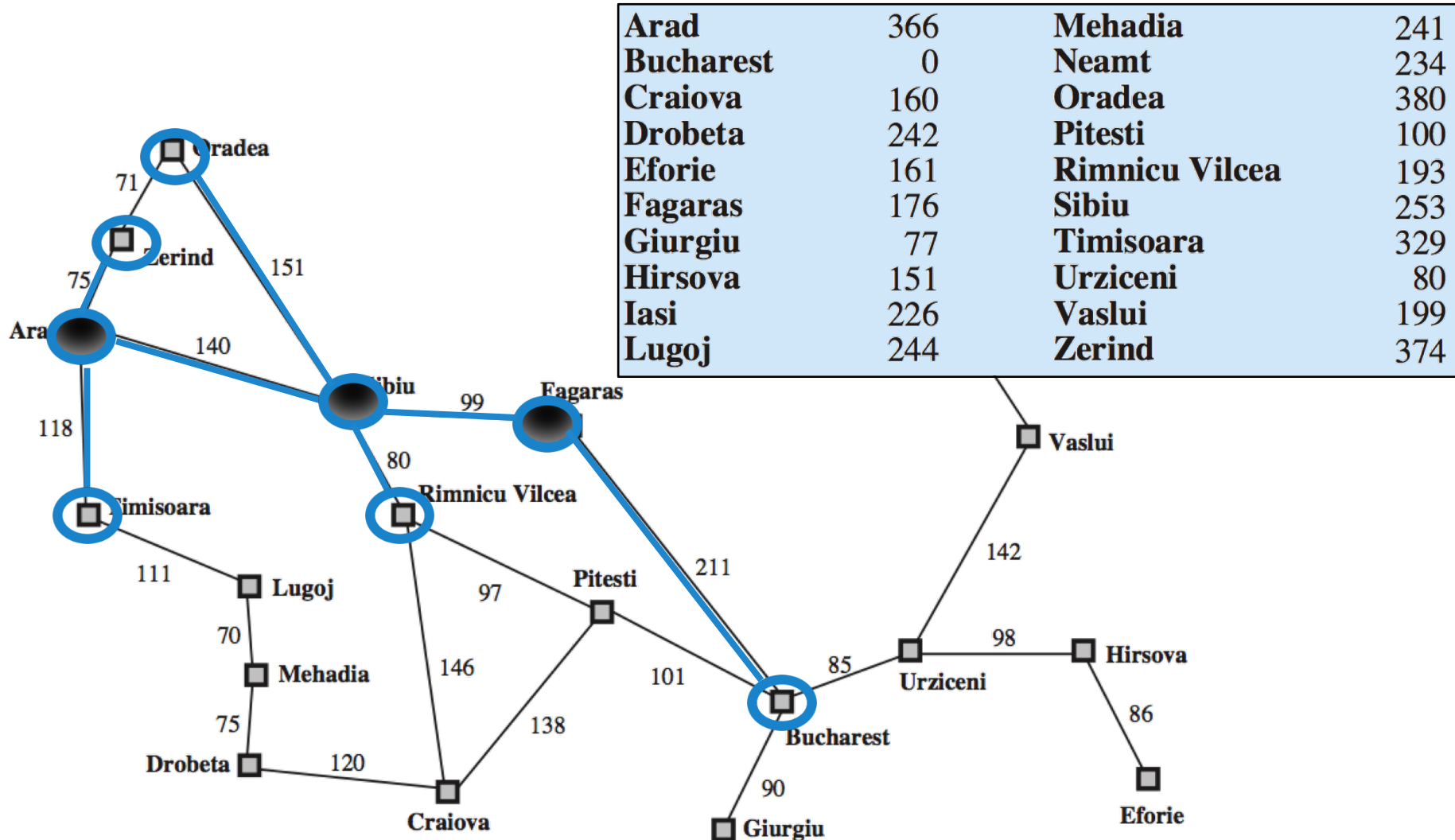
Ejemplo Algoritmo voraz (en árbol)



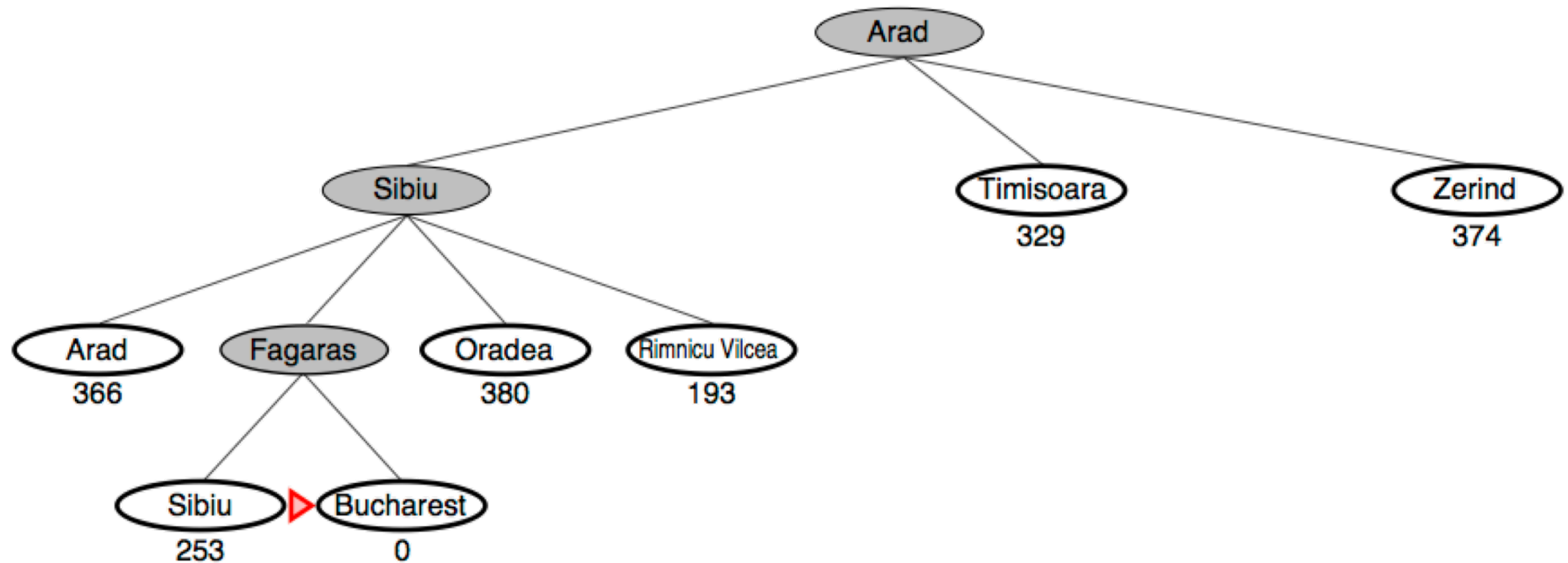
Ejemplo Algoritmo voraz (en árbol)



Ejemplo Algoritmo voraz (en árbol)

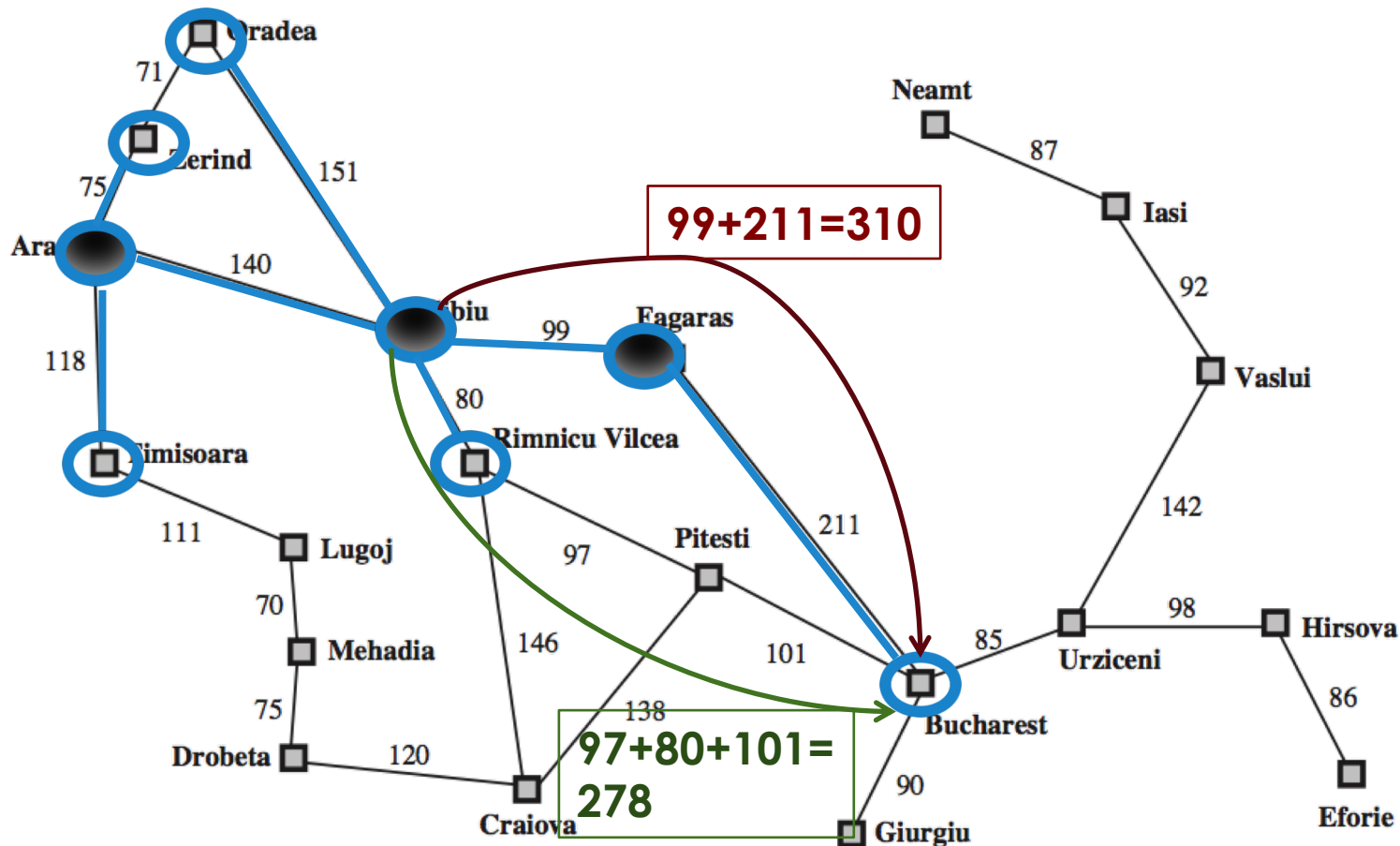


Ejemplo Algoritmo voraz (en árbol)



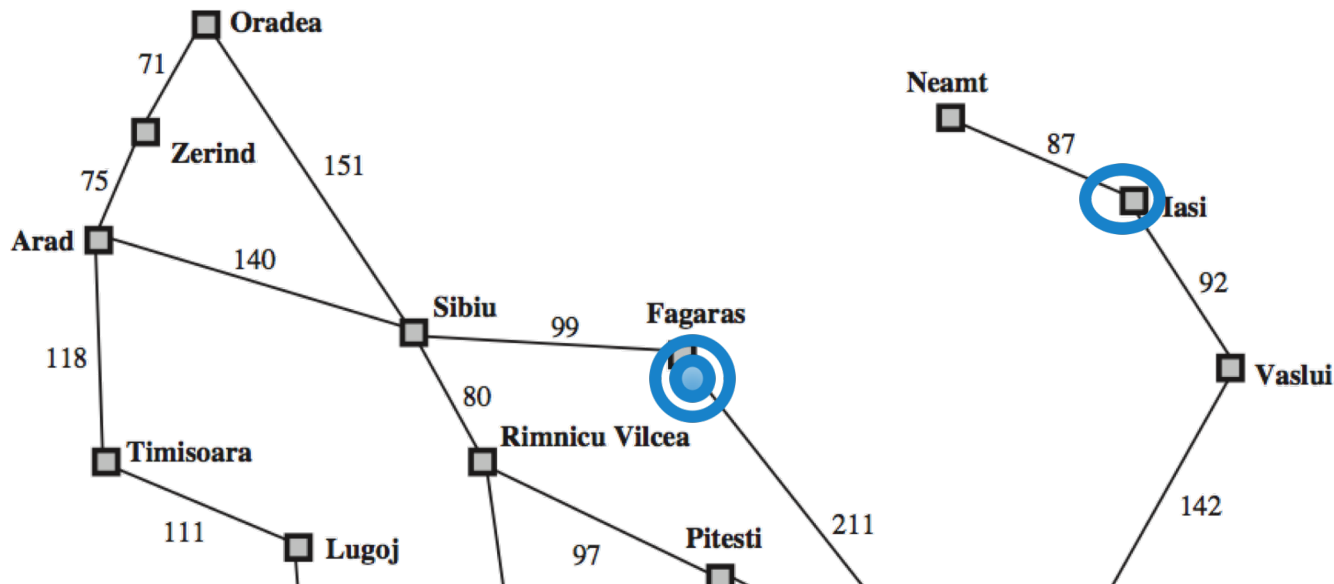
Evaluación: ¿Óptimo?

- Óptimo: NO.



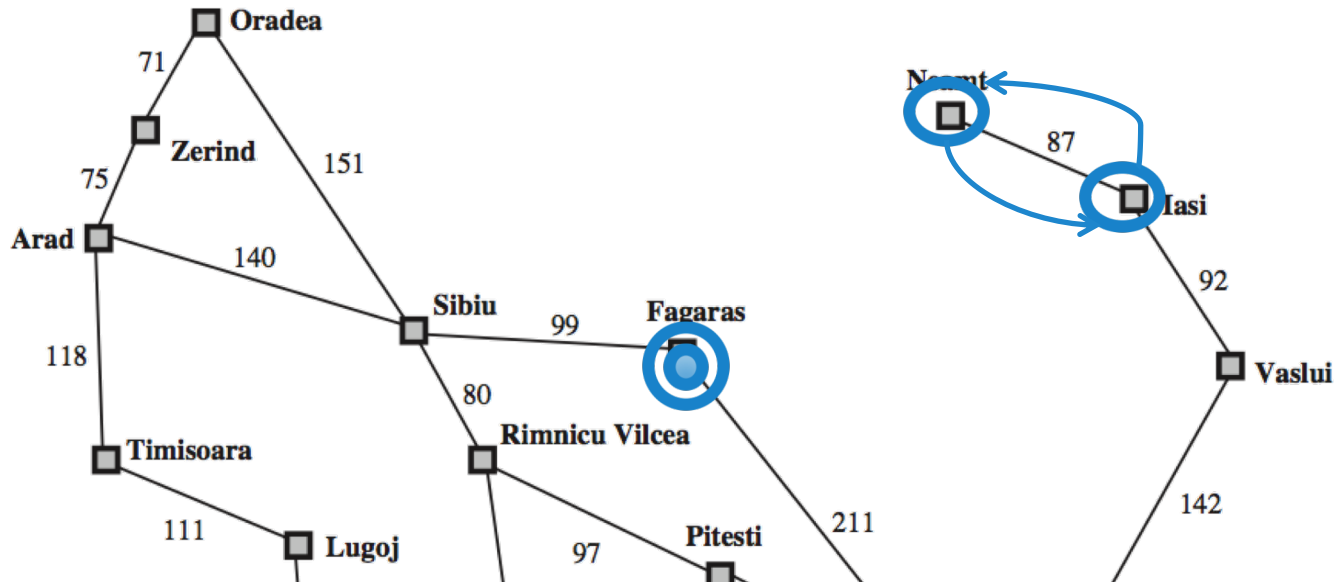
Evaluación algoritmo voraz

- Óptimo: NO
- Completo:
 - No en la búsqueda en árbol. Incluso con un espacio de estados finito, puede entrar en un bucle infinito.



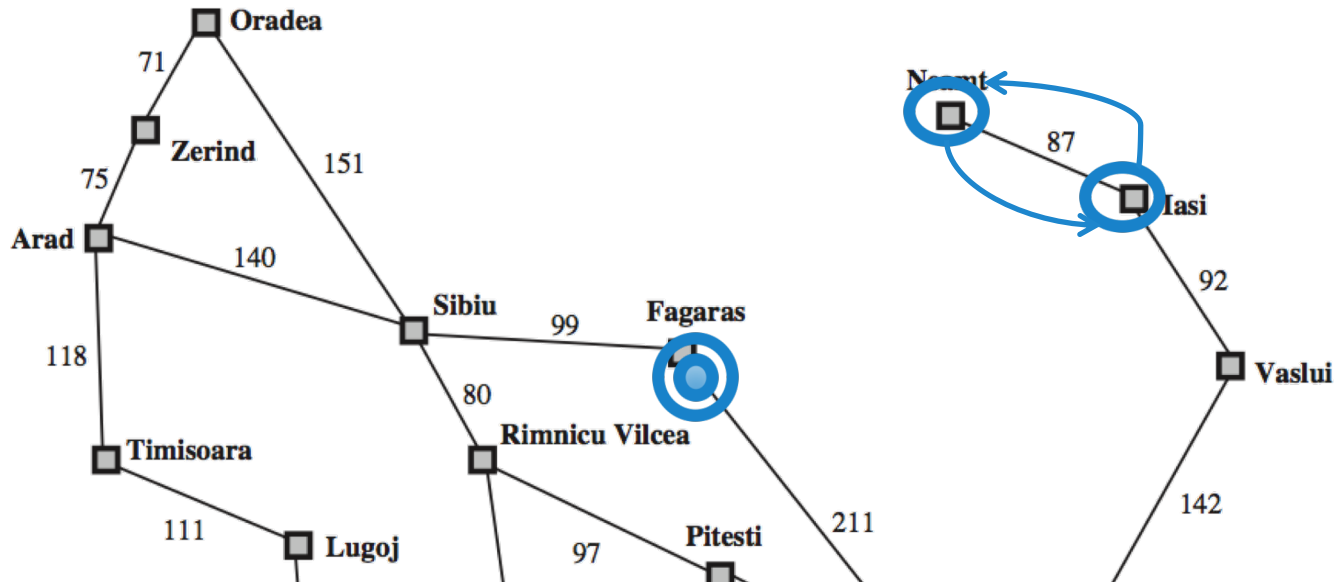
Evaluación algoritmo voraz

- Óptimo: NO
- Completo:
 - No en la búsqueda en árbol. Incluso con un espacio de estados finito, puede entrar en un bucle infinito.



Evaluación algoritmo voraz

- Óptimo: NO
- Completo:
 - No en la búsqueda en árbol.
 - Si en búsqueda en grafo si el espacio es finito.



Búsqueda A*

Idea: evitar expandir caminos que ya tienen caminos costosos.

- Función de evaluación $f(n) = g(n) + h(n)$

$g(n)$ = coste para alcanzar el nodo n

$h(n)$ = estimación del coste para alcanzar el objetivo desde el nodo n .

$f(n)$ = estimación del coste total del camino desde n al objetivo.

Algoritmo idéntico a la búsqueda de coste uniforme, utilizando **f** en lugar de **g** .

Búsqueda A*

La búsqueda A* es optima y completa si la función heurística cumple ciertas propiedades (admisibilidad y consistencia)

- Una heurística es **admisibile** si nunca sobreestima el coste real de alcanzar el objetivo.
- Una heurística admisible es **optimista**

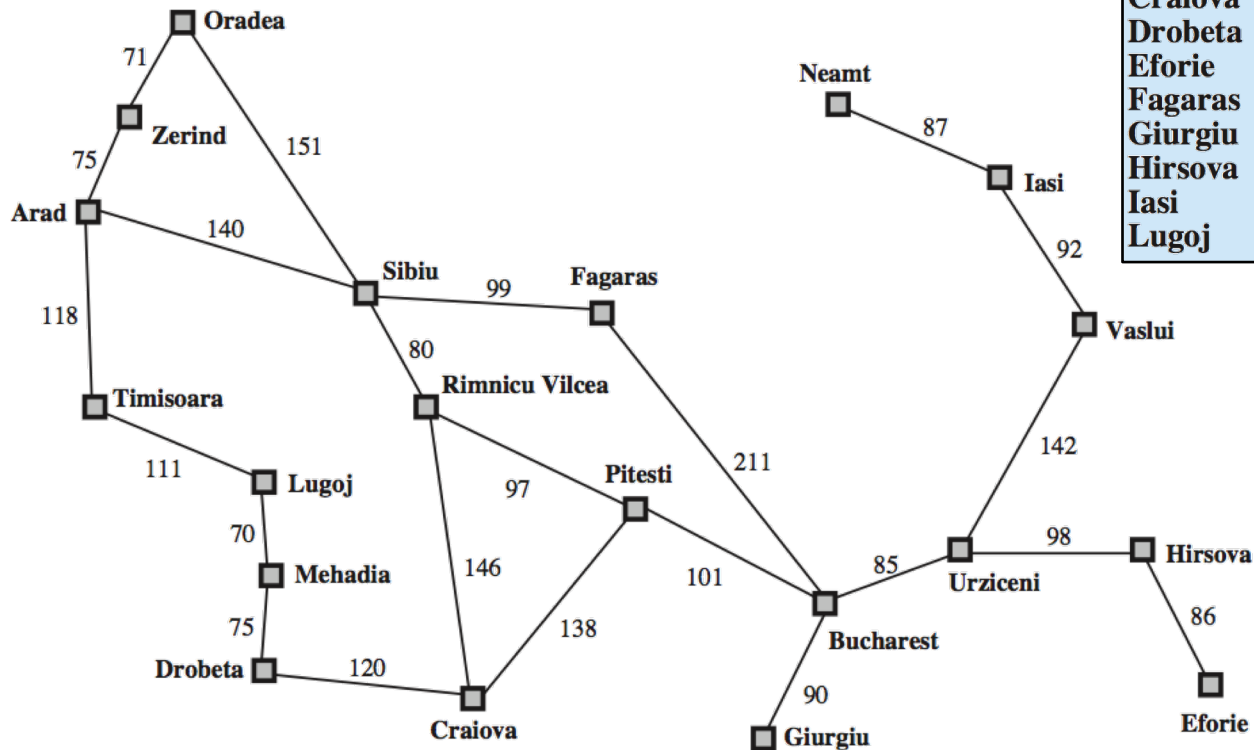
Formalmente:

1. $h(n) \leq h^*(n)$ donde $h^*(n)$ es el coste **real** de n
2. $h(n) \geq 0$, de forma que $h(G)=0$ para cualquier objetivo
p.e. $h_{SLD}(n)$ nunca sobreestima la distancia real a recorrer.

Ejemplo Rumanía: A^* y h_{SLD}

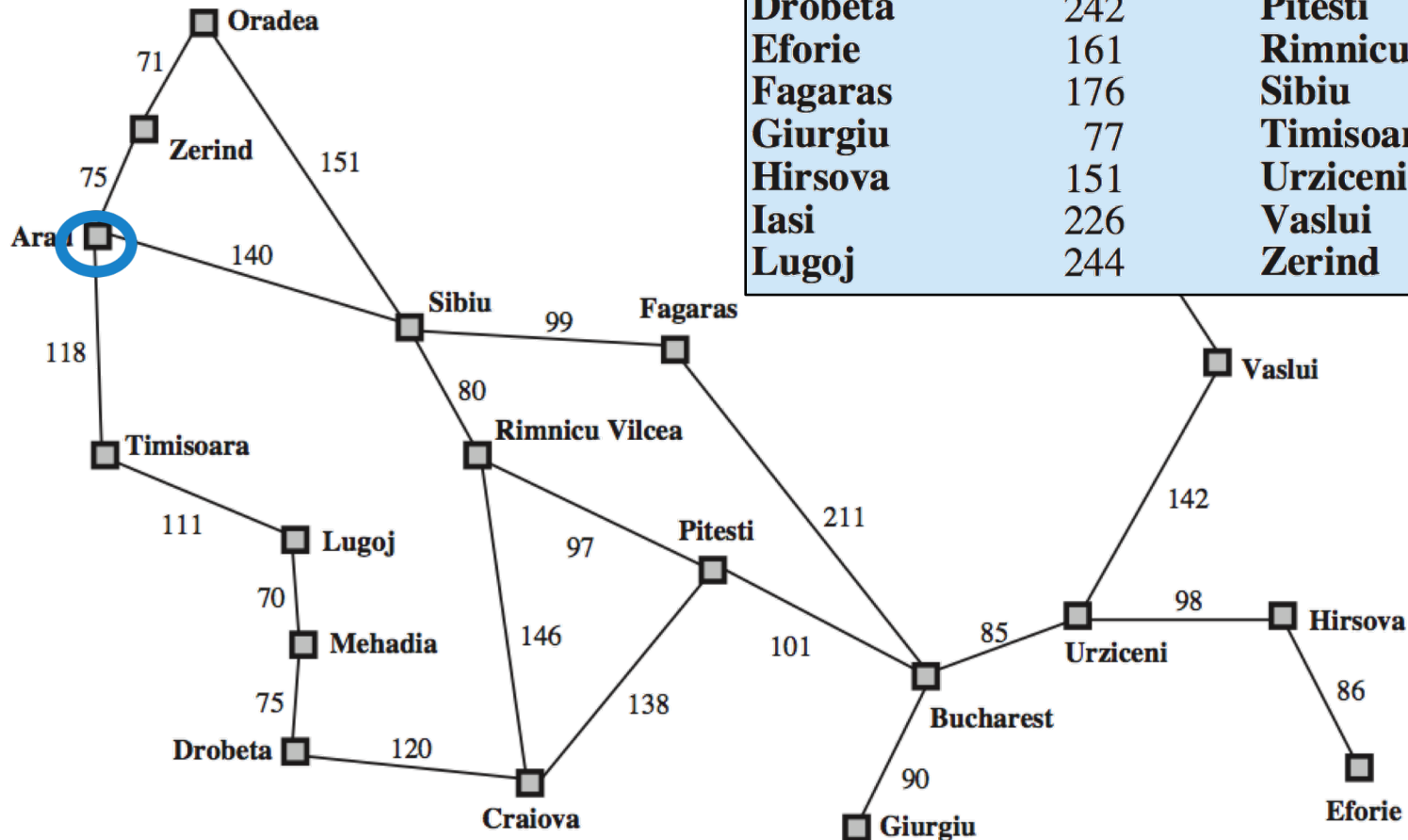
Queremos viajar de Arad a Bucharest.

- Estado Inicial = Arad



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Ejemplo Rumanía: A^* y h_{SLD}

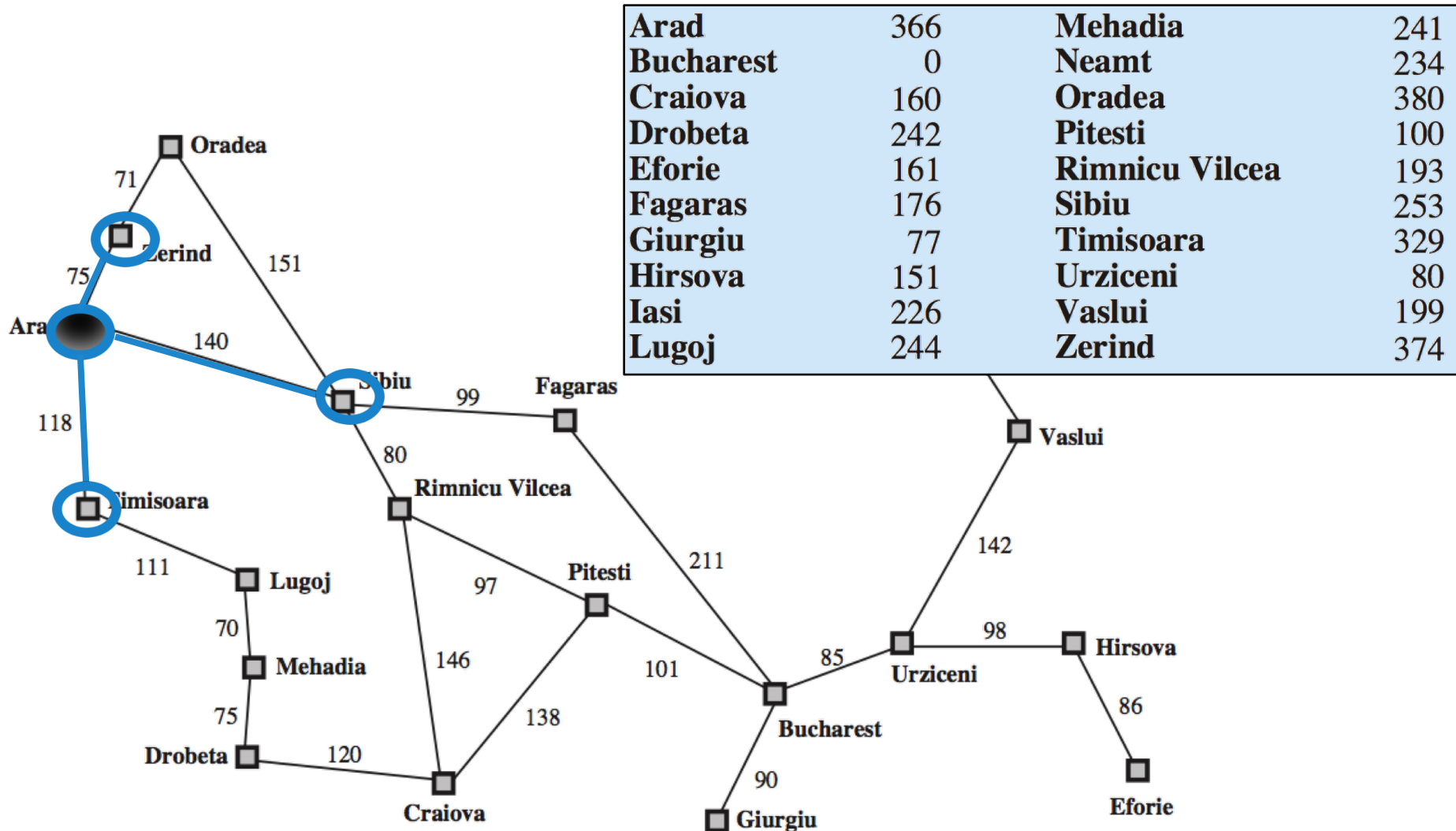


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

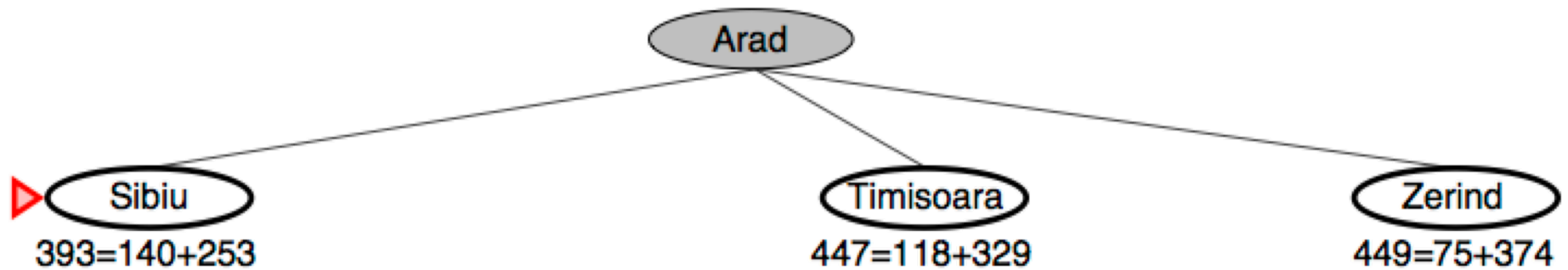
Ejemplo Rumanía: A^* y h_{SLD}

▶ Arad
 $366=0+366$

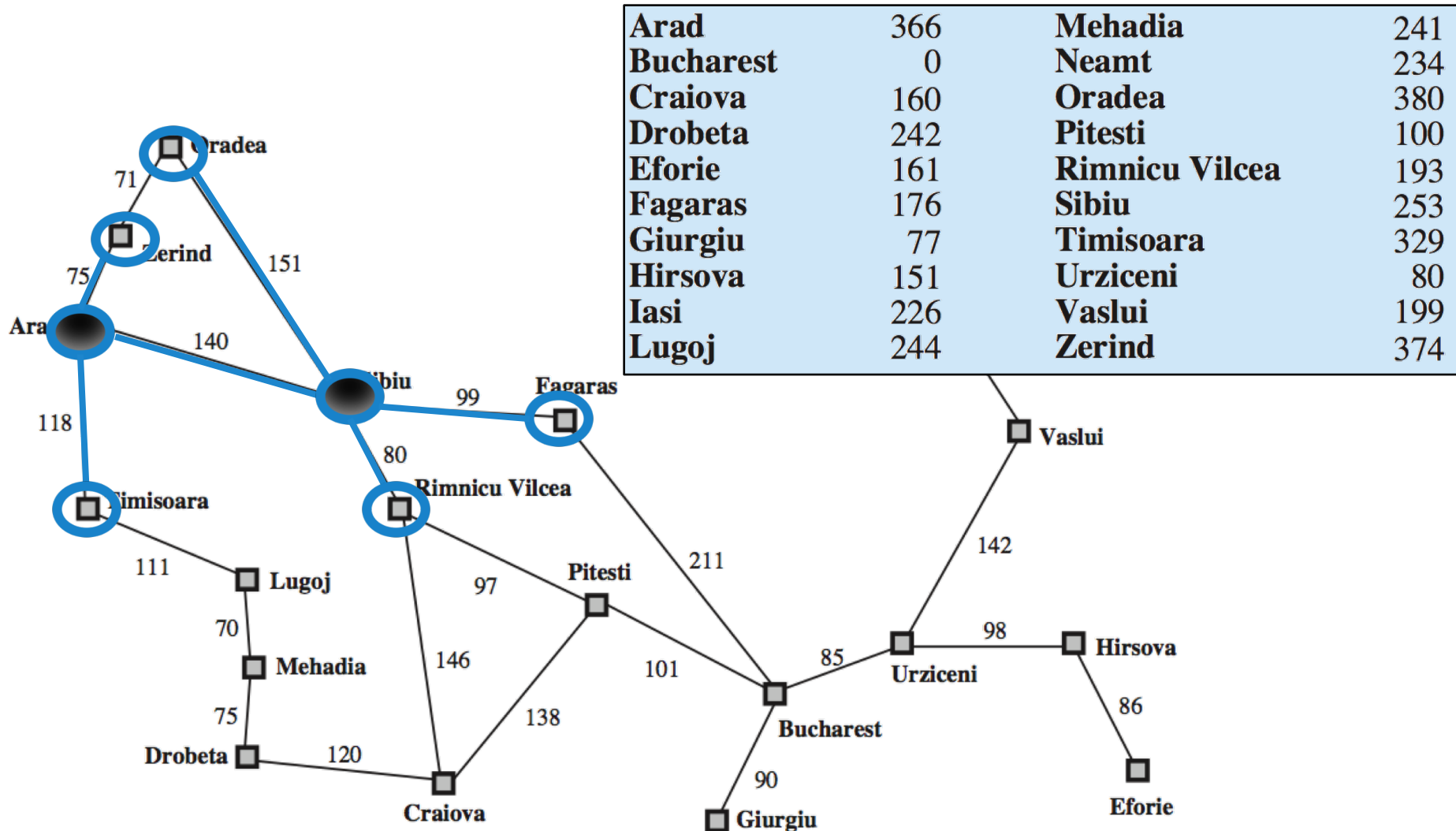
Ejemplo Rumanía: A^* y h_{SLD}



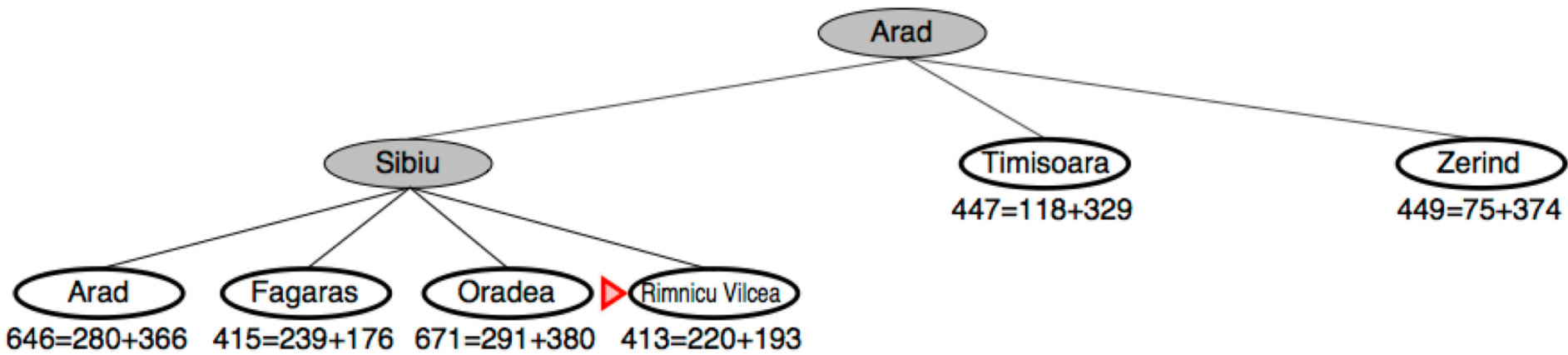
Ejemplo Rumanía: A^* y h_{SLD}



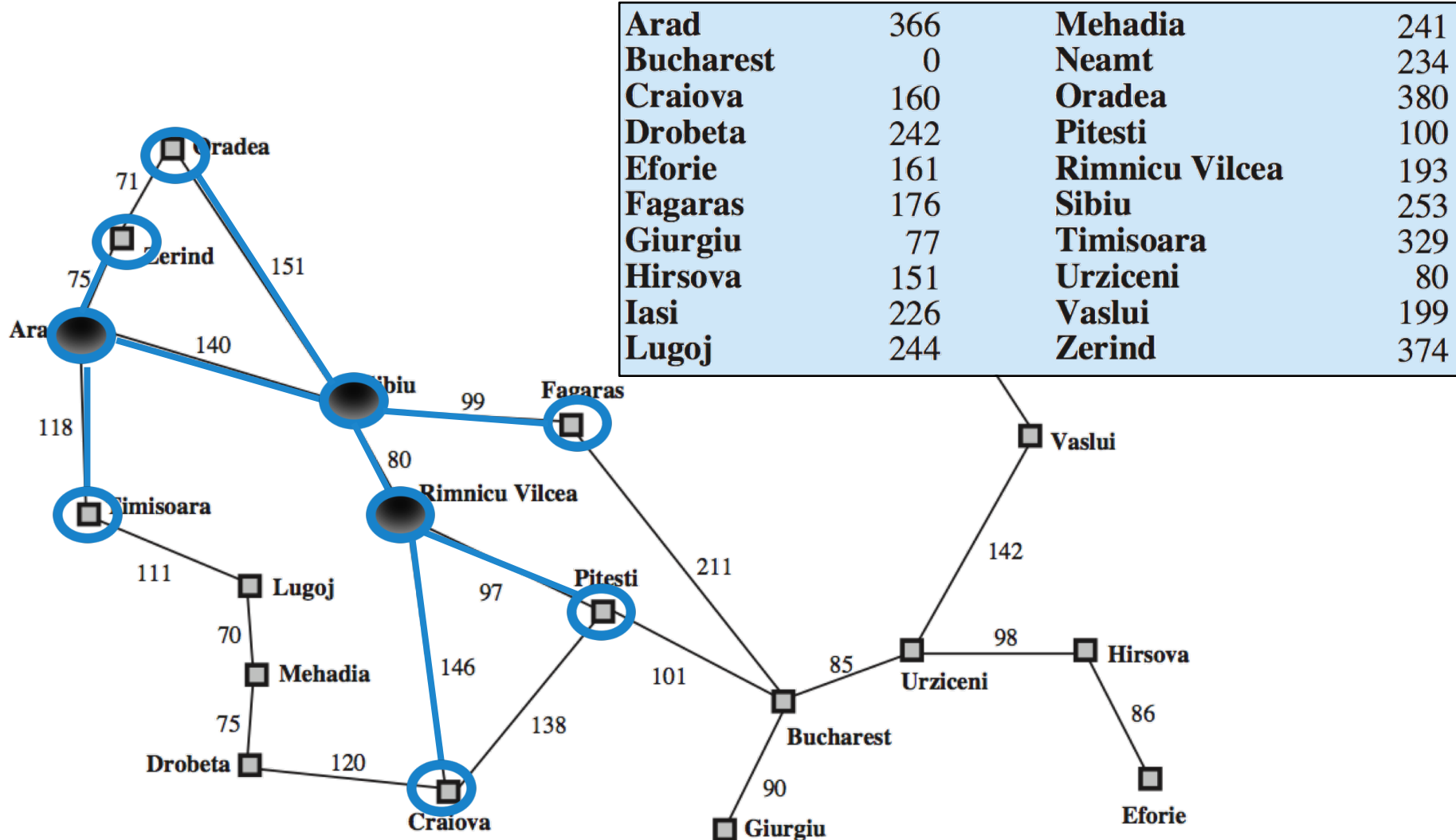
Ejemplo Rumanía: A^* y h_{SLD}



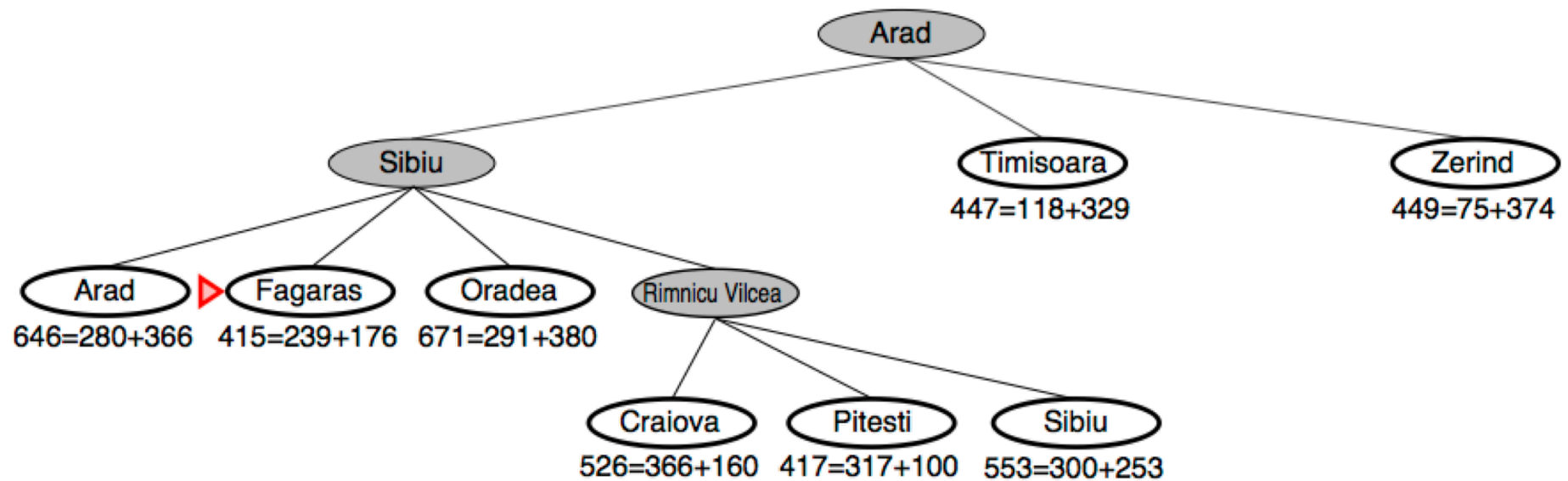
Ejemplo Rumanía: A^* y h_{SLD}



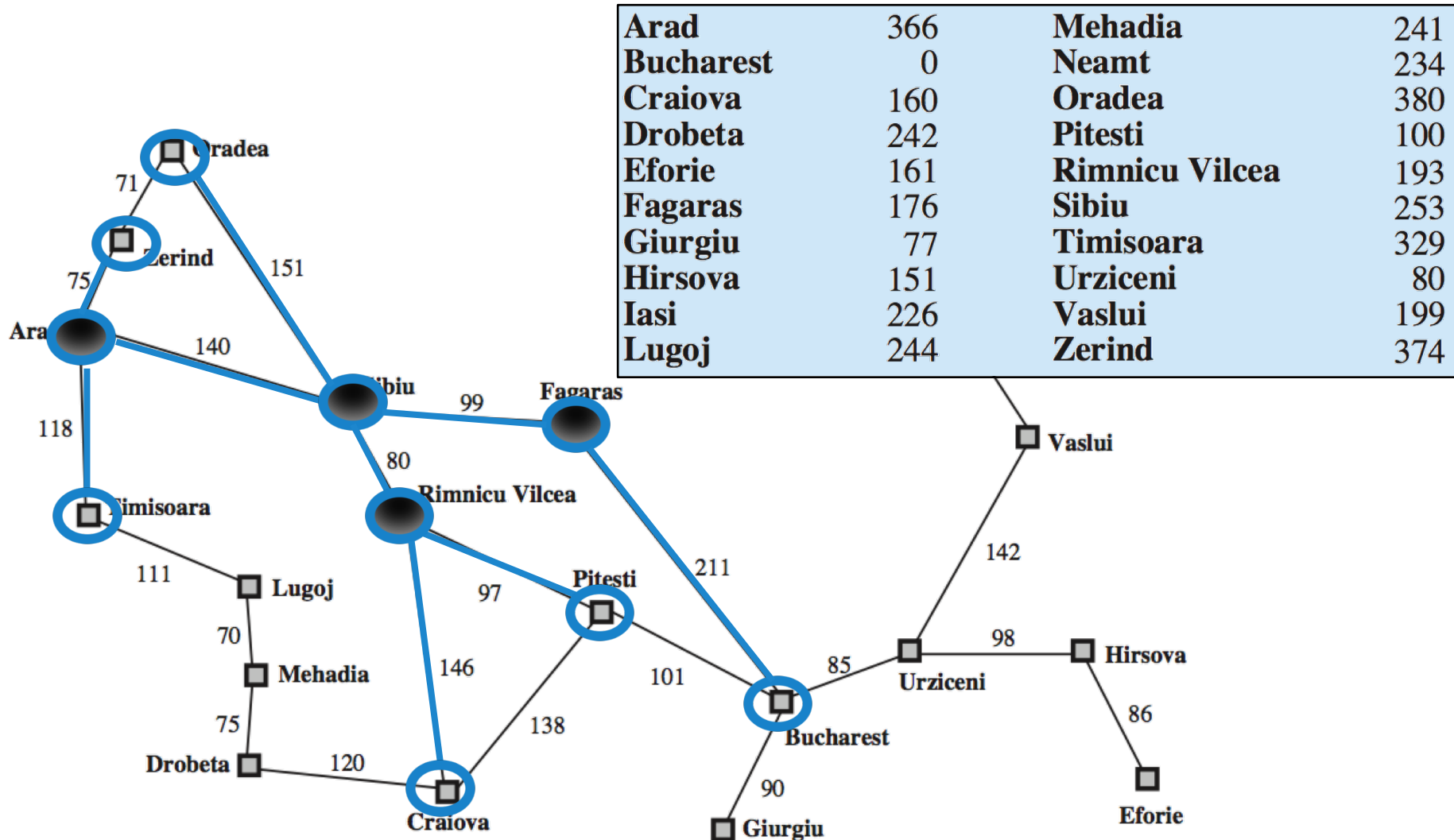
Ejemplo Rumanía: A^* y h_{SLD}



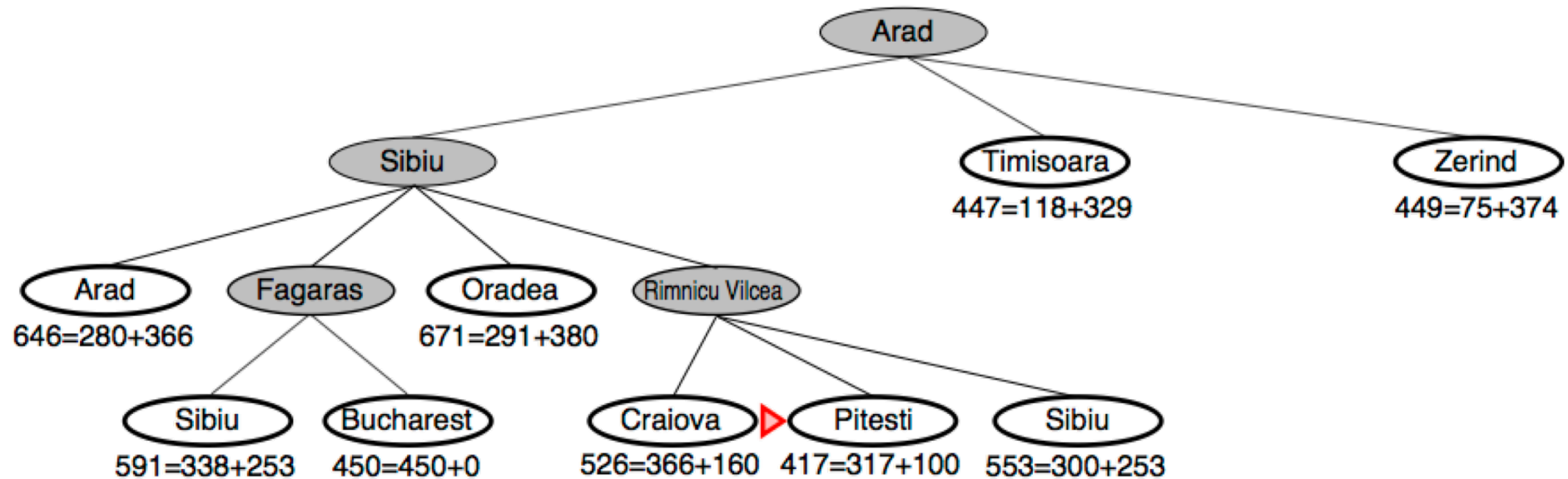
Ejemplo Rumanía: A^* y h_{SLD}



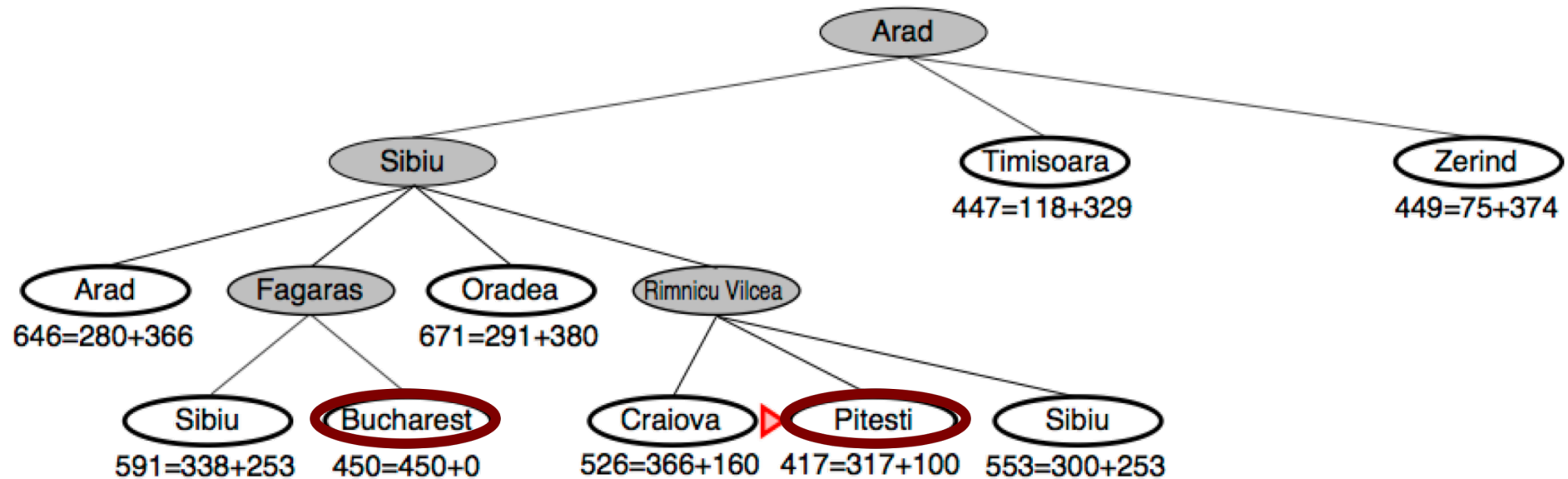
Ejemplo Rumanía: A^* y h_{SLD}



Ejemplo Rumanía: A^* y h_{SLD}



Ejemplo Rumanía: A^* y h_{SLD}

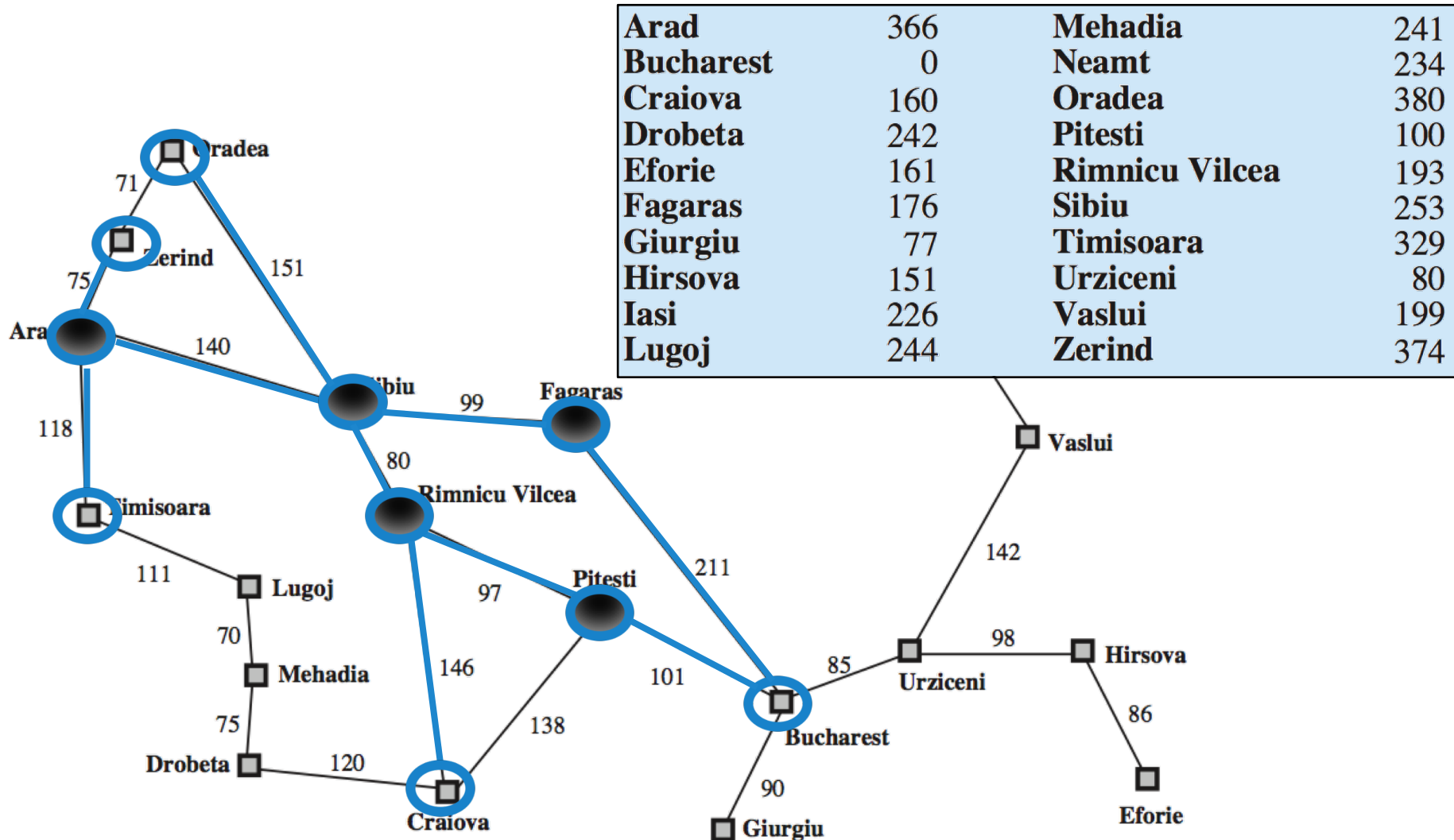


Aparece Bucharest en la frontera, pero con $f=450$.

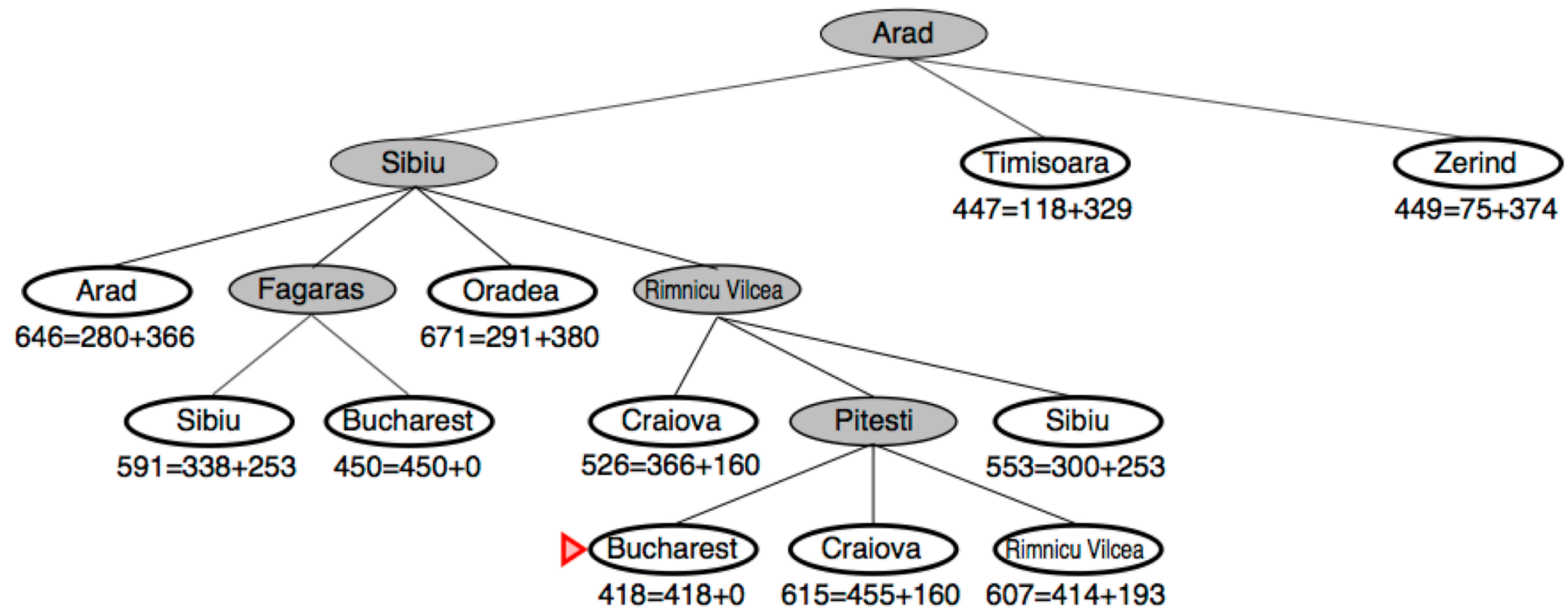
Pitesti tiene el menor valor de f 417,

podríamos encontrar una solución de menor coste a través de Pitesti

Ejemplo Rumanía: A^* y h_{SLD}



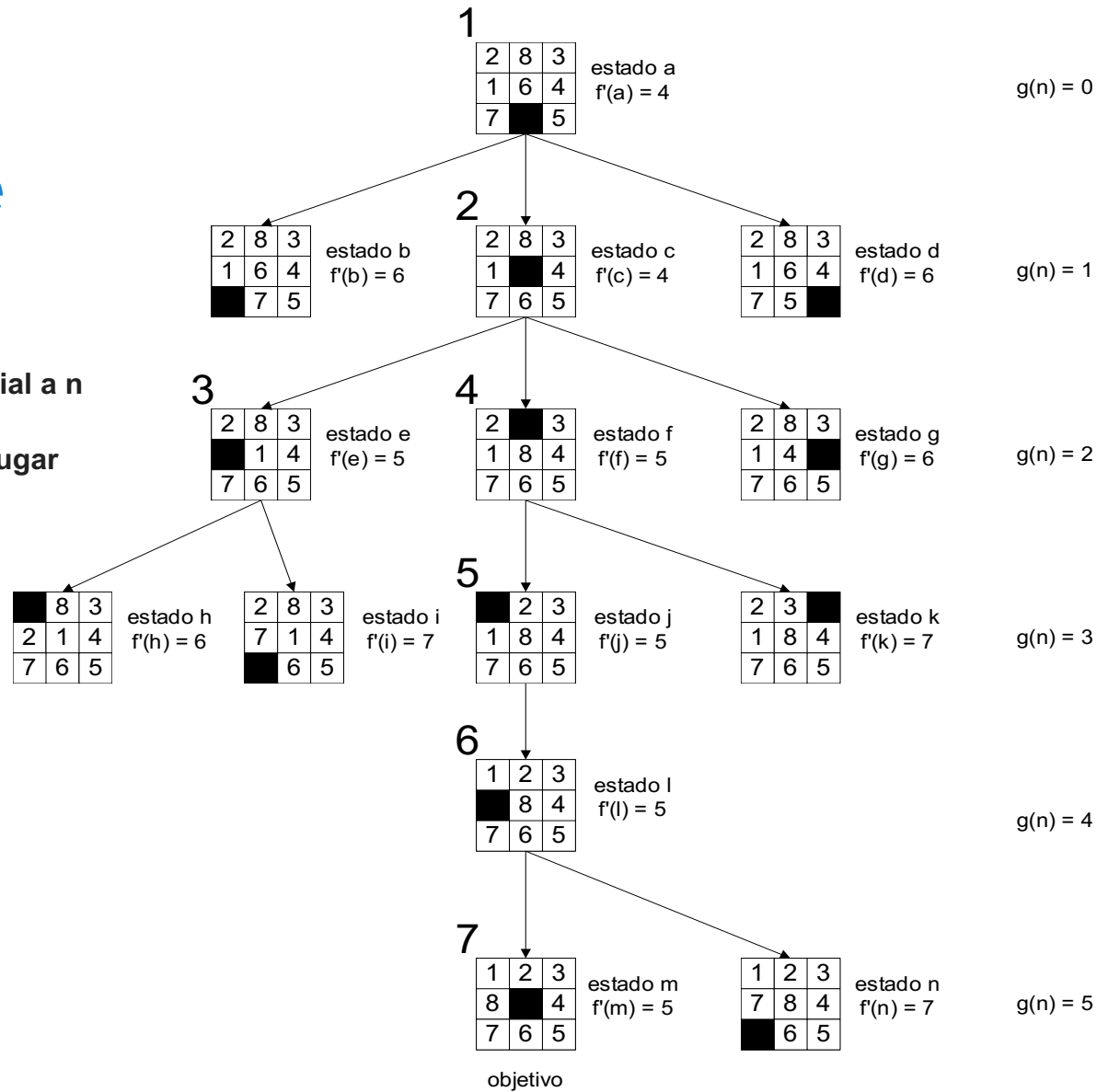
Ejemplo Rumanía: A^* y h_{SLD}



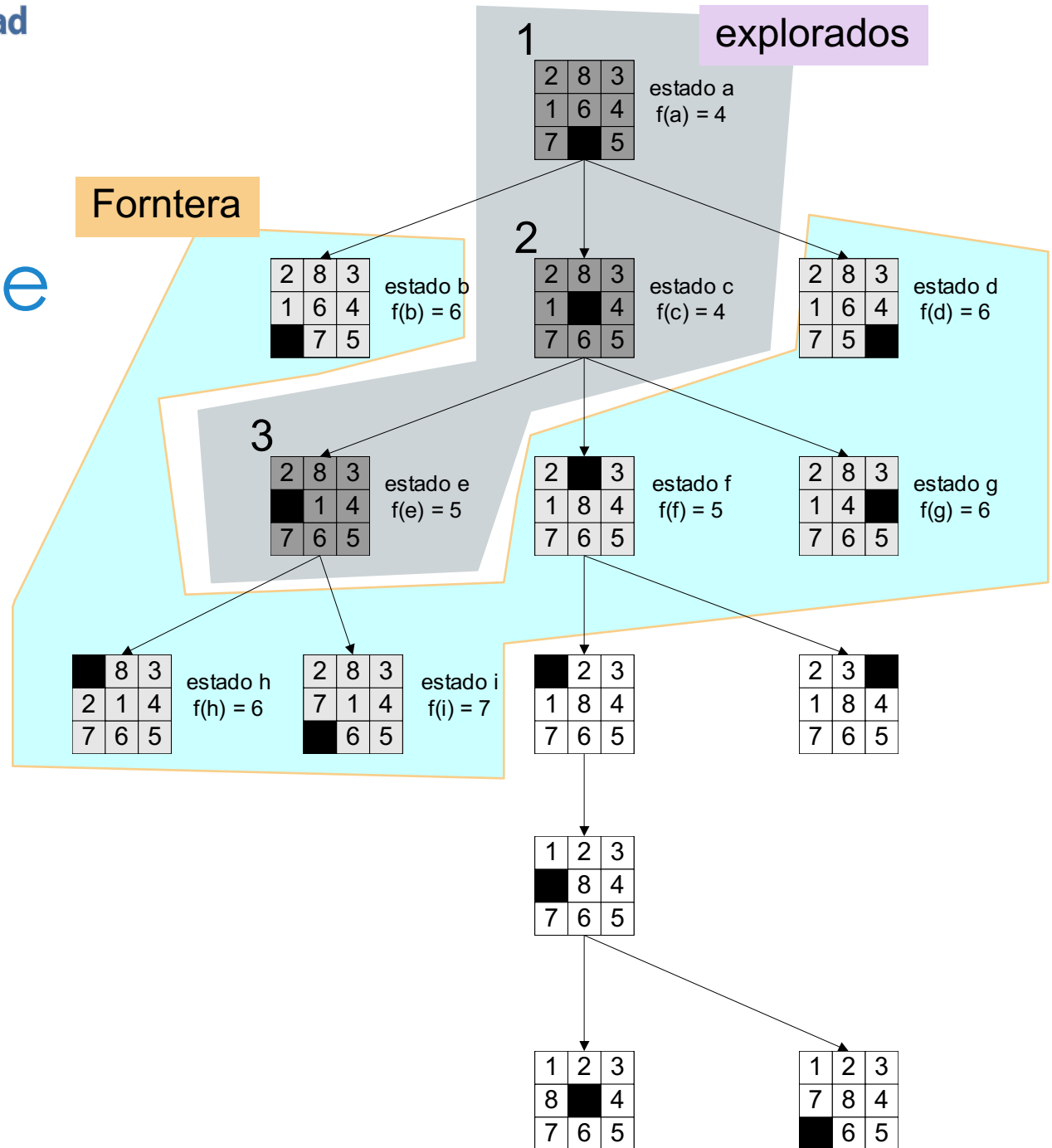
- A^* en árbol: Solución óptima si $h(n)$ es admisible

A*/8-puzzle

- $f'(n) = g(n) + h'(n)$
- $g(n)$ = distancia real del estado inicial a n
- $h'(n)$ = número de piezas fuera de lugar



A*/8-puzzle



Consistencia

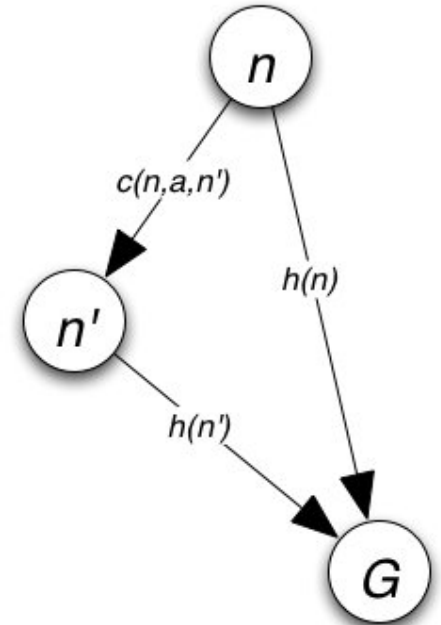
- Una heurística es consistente si

$$h(n) \leq c(n, a, n') + h(n')$$

- Si h es consistente

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \\ &\geq f(n) \end{aligned}$$

por lo tanto $f(n)$ es no decreciente a lo largo de cualquier camino.

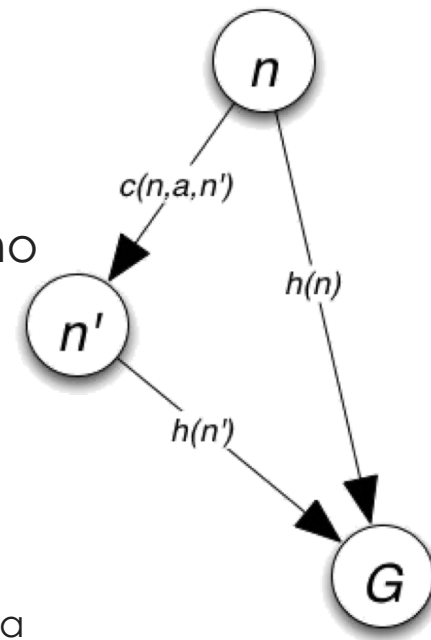


A* Búsqueda en grafo

1. $f(n)$ es no decreciente a lo largo de cualquier camino

$$f(n') \geq f(n)$$

2. Cuando se selecciona un nodo n para expansión, el camino óptimo a este nodo ha sido encontrado
 - Si no fuera el caso, habría otro nodo n' en la frontera por el que pasaría el camino óptimo desde el origen al nodo n . Por ser $f(n)$ una función no decreciente, $f(n')$ tendría menor valor y tendría que haber sido seleccionado primero.

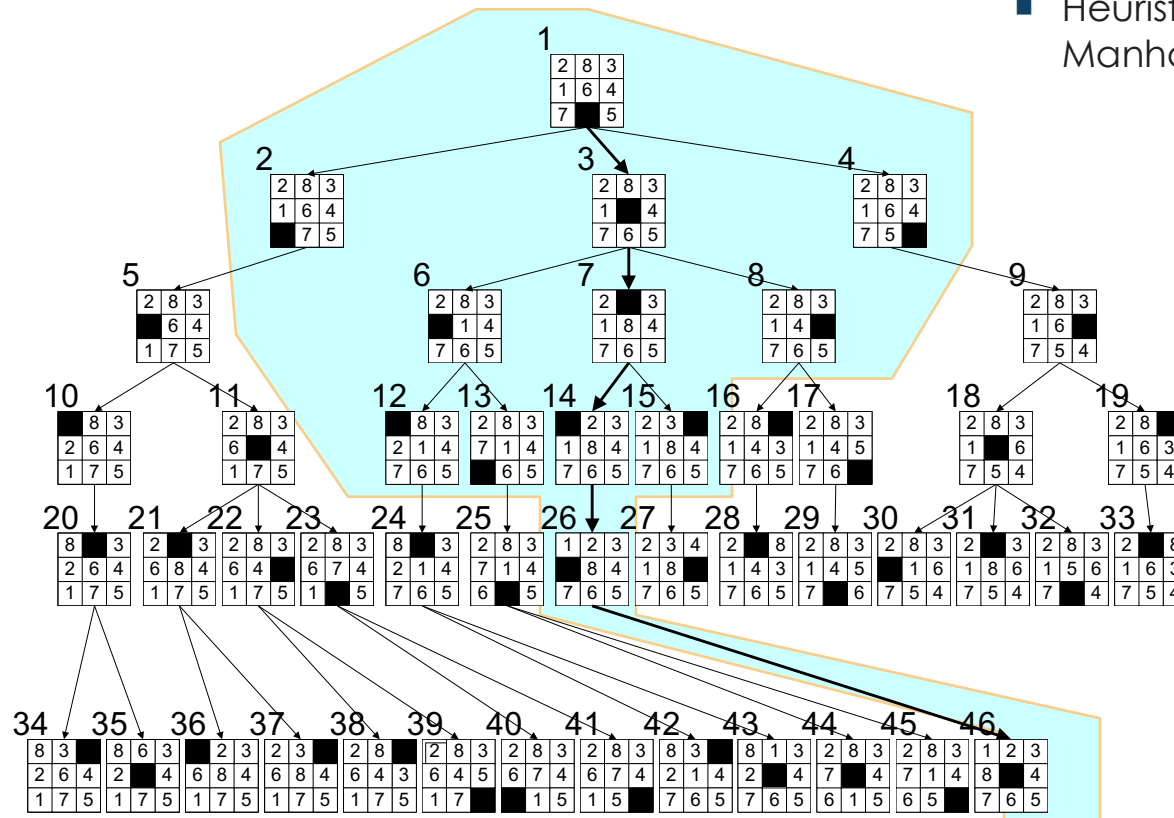


De las dos observaciones precedentes se sigue que **una secuencia de nodos expandidos por A* utilizando una búsqueda en grafo es en orden no decreciente de $f(n)$.**

Comparación A^* y anchura en 8-puzzle

- Comparación del espacio de búsqueda

- Comparación del espacio de estados entre los algoritmos A* y primero en anchura
 - Heurística utilizada: distancia Manhattan



Ciencia e Ingeniería en el estudio de la mente –

Búsqueda Informada

Luis F. Castillo – Manuel G. Bedia

